

REPORT

Course: Operating Systems

Assignment: Assignment 1 – Character Device Driver with Version Validation and Timed Tasks

Submitted by:

Name: Kona Bhuvaneswar Reddy

Roll Number: B240214CS

Batch: CS02

Date: April 8, 2026

Problem Statement

Objective: Developing a Linux Kernel Module (Character Device Driver) that:

- Validates kernel version compatibility upon insertion.
- Implements a timer-constrained task sequence.
- Enforces a strict order of operations (Read then Write).
- Logs status and results via the kernel ring buffer (`dmesg`).

Methodology

The driver is implemented using a modular approach, separating the module's lifecycle management (`main.c`) from the character device's file operations (`device_file.c`).

- **Kernel Version Validation:** Uses the `LINUX_VERSION_CODE` macro. User input is captured via `module_param_array` and compared against the compiled kernel version.
- **Concurrency & Timing:** Implements a kernel timer (`timer_list`) to track the grace period and a wait queue (`read_wait`) for process synchronization.
- **State Machine:** Uses a set of boolean flags (`read_done`, `write_done`, `order`) to track user interaction from user-space via the Virtual File System (VFS).
- **Data transfer:** Uses `copy_to_user` and `copy_from_user` for secure boundary crossing between kernel and user space.

Implementation

Tools used:

- **Language:** C (Kernel-space).
- **Environment:** Linux (Ubuntu) with Kernel headers.

A. Module Parameters & Version Check

The driver accepts parameters at load time: `kernel_version` (array) and `timer` (renamed from `time` via `module_param_named`). Upon calling `insmod`, the `simple_driver_init` function extracts the system's kernel version using bitwise operations:

- **Major Version:** `(LINUX_VERSION_CODE >> 16) & 0xFF`
- **Minor Version:** `(LINUX_VERSION_CODE >> 8) & 0xFF`

If the user-provided parameters do not match these values, the module returns `-EINVAL`, preventing the driver from being loaded into the kernel.

B. Device Registration

To ensure the device is accessible as `/dev/my_dev`, the driver performs:

1. **Dynamic Allocation:** `alloc_chrdev_region` is used to request a major number dynamically, avoiding conflicts with existing drivers.
2. **CDEV Initialization:** `cdev_init` links the character device to the `file_operations` structure, mapping `read` and `write` syscalls to our internal functions (`device_file_write` & `device_file_read`).
3. **Device Node Creation:** `class_create` and `device_create` interface with `udev` to automatically generate the `/dev/my_dev` file node with the correct permissions.

C. Timer & Task Logic

The Timer: `timer_setup` initializes the `my_timer` structure with `my_timer_callback`. The countdown begins when `mod_timer` is called with `jiffies + msecs_to_jiffies(time * 1000)`.

The Read-Write Order:

- In `device_file_read`, the flag `read_done` is set to true.
- In `device_file_write`, the driver checks `read_done`. If it is false, the `order` flag is set to **false**, marking the task as a failure even if performed within the time limit.
- If both actions are completed in order, `del_timer` is called to stop the clock and confirm success.

D. Module Removal

The `__exit` function acts as the final validator. It evaluates the boolean flags:

- If `read_done`, `write_done`, and `order` are true while `timer_expired` is false, it logs a **Success** message along with the captured username.
- Otherwise, it logs specific **Failure** messages ("Order violated" or "Time Expired").
- The **Success** and **Failure** messages will be printed to `dmesg`.
- Finally, `unregister_device` cleans up the `cdev`, class, and allocated regions.

Execution Procedure

Compilation: Run `make` to generate `simple-module.ko`.

Insertion: `sudo insmod simple-module.ko kernel_version=6,8 timer=30`

(Adjust versions based on your current `uname -r`).

Module Info: `modinfo simple-module.ko`

Check Logs: `dmesg | tail`.

Read Action: `sudo cat /dev/my_dev`.

Write Action: `sudo echo $(whoami) > /dev/my_dev`.

Removal: `sudo rmmod mymodule`.

Final Result: `dmesg | tail`.

Screenshots

Screenshot 1: Successful Completion. Should show `dmesg` outputting the "Successfully completed the actions within time" message with your username.

```
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo insmod simple-module.ko kernel_version=6,8 timer=30
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo cat /dev/my_dev
Write your username.
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ echo $(whoami) | sudo tee /dev/my_dev
user
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo rmmod simple_module
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo dmesg
[ 310.828621] Simple-driver: register_cdev() is called.
[ 310.828730] Simple-driver: Registered with Major = 235, Minor = 0, Timer = 30
[ 314.862017] Simple-driver: Username 'user' recorded.
[ 314.862021] Simple-driver: Actions completed! Timer stopped successfully.
[ 317.477415] Simple-driver: Successfully completed the actions within time. User: user
[ 317.477418] Simple-driver: unregister_device() is called
[ 317.477533] Simple-driver: Unregistered
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$
```

Screenshot 2: Version Mismatch. Show the terminal error for `insmod` and the `dmesg` log: "Version mismatch!".

```
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo insmod simple-module.ko kernel_version=6,5 timer=30
insmod: ERROR: could not insert module simple-module.ko: Invalid parameters
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo dmesg
[ 39406.365929] Simple-driver: Version mismatch! Kernel is 6.8
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$
```

Screenshot 3: Order Violation. Show `dmesg` logs where a write was attempted before a read, resulting in "ERROR - Write attempted before Read."

```
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo insmod simple-module.ko kernel_version=6,8 timer=30
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ echo $(whoami) | sudo tee /dev/my_dev
user
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo cat /dev/my_dev
Write your username.
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo rmmod simple_module
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo dmesg
[ 865.924600] Simple-driver: register_cdev() is called.
[ 865.924693] Simple-driver: Registered with Major = 235, Minor = 0, Timer = 30
[ 869.808848] Simple-driver: ERROR - Write attempted before Read. Order violated!
[ 869.808852] Simple-driver: Username 'user' recorded.
[ 869.808853] Simple-driver: Actions completed! Timer stopped successfully.
[ 878.517480] Simple-driver: Task failure: Actions not completed in order.
[ 878.517496] Simple-driver: unregister_device() is called
[ 878.517645] Simple-driver: Unregistered
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$
```

Screenshot 4: Timeout Case. Show that after the timer expires, the module exit prints "Task failure: Time Expired."

```
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo insmod simple-module.ko kernel_version=6,8 timer=10
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo cat /dev/my_dev
Write your username.
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ echo $(whoami) | sudo tee /dev/my_dev
user
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo rmmod simple_module
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$ sudo dmesg
[ 918.013989] Simple-driver: register_cdev() is called.
[ 918.014139] Simple-driver: Registered with Major = 235, Minor = 0, Timer = 10
[ 928.467781] Simple-driver: Timer expired!
[ 930.110385] Simple-driver: Username 'user' recorded.
[ 930.110389] Simple-driver: Actions completed, but timer had already expired.
[ 936.271424] Simple-driver: Task failure: Time Expired.
[ 936.271427] Simple-driver: unregister_device() is called
[ 936.271551] Simple-driver: Unregistered
user@Kona-ThinkPad-T14-Gen-1:~/Desktop/OS_Assignment/KONA_B240214CS$
```

Conclusion

The implementation successfully demonstrates the interaction between user-space requests and kernel-space drivers. By utilizing `LINUX_VERSION_CODE`, we ensured environment compatibility, while the use of kernel timers and state flags provided a robust way to enforce task deadlines and operational sequencing. This assignment highlights the importance of state management within the Linux kernel to maintain data integrity and operational flow.